Laborationsrapport

Projektuppgift
DT211G, Frontend

Författare: Elin Ronda, elro2506@student.miun.se
Termin, år: VT, 2026

Mittuniversitetet
MID SWEDEN UNIVERSITY

1 Sammanfattning

Här sammanfattar du ditt arbete. Tanken är att man ska kunna läsa denna sektion, och snabbt förstå vad resten av rapporten handlar om. Tänk lite som baksidan av en bok.

2 Innehållsförteckning

.....1
1 Sammanfattning.....2
2 Innehållsförteckning.....3
3 Introduktion.....4
3.1 Solspaning.....4
3.2 Inspiration och research.....4
3.3 Krav för Solspaning.....4
4 Teori.....6
5 Metod.....9
5.1 Open-Meteo.....9
5.2 Leaflet.....9
5.3 Logotyp och ikoner.....9
5.4 Grafisk profil.....9
5.5 SCSS.....9
5.5.1 Partials.....9
5.6 JavaScript.....10
5.6.1 AJAX..... **Fel! Bokmärket är inte definierat.**
6 Konstruktion.....11
7 Resultat.....15
8 Slutsatser.....16
9 Källförteckning.....17
10 Bilagor.....19

3 Introduktion

I detta projekt ska jag skapa en hemsida som ser ut som en webbapplikation. Denna webbapplikation ska vara en så kallad mashup, som är en slags hybrid som möjliggör kombinationen av olika funktioner från olika källor (1). Med mashups finns flera olika möjligheter att skapa webbplatser eller applikationer som möjliggör kreativa funktioner för användarna, men webbutvecklare är beroende av att leverantören av tjänster och API som mashupen använder sig av alltid är tillgängliga och funktionella.

I projektuppgiftens beskrivning fanns bland annat ett exempel på mashup med: ”en webbtjänst för kartor [som] används för att visa väder för specifika platser från en vädertjänst”. Jag valde att göra en liten twist på detta, och att skapa webbplatsen Solspaning.

3.1 Vad är Solspaning?

På Solspaning ska användare mötas av ett tydligt budskap med frågeställning: att kunna välja vilket väder man önskar ha för dagen, och på så sätt i en karta få se var i Sverige önskat väder finns. Detta är både en rolig idé i sig, men också ett sätt för användare att eventuellt upptäcka nya städer i sitt närområde.

3.2 Inspiration och research

Jag har sökt mig runt på olika befintliga vädertjänster, så som Klart.se (2) och genom att titta på TV4-vädret (3). Där har jag bland annat sett att det inte finns någon anledning att visa vädret för alla landets städer, och att det därför räcker med ungefär 15 städer för att täcka upp de flesta områden och ändå ge en rimlig bild över vilket väder som finns.

3.3 Krav för Solspaning

Jag har gjort en prototyp i Figma, se figur 1 och 2 i bilagor för desktopläge respektive responsivt läge. I prototypen går det att se tänkt utformning av min webbapplikation, med en stor, centrerad logotyp följt av någon form av text som frågar användaren vilket väder den önskar se just nu. Enligt prototypen är texten ”Hej, vilket väder vill du ha idag?” men det kan komma att ändras om jag finner en mer passande text.

Under den texten så ska det finnas 4 ikoner föreställande sol, regn, vind och moln med tillhörande texter som ska förtydliga ikonerna ur ett tillgänglighetsperspektiv. Vid klick på någon av ikonerna ska man automatiskt scrollas ner till kartan under och där se utplacerade ikoner på de städer som har det efterfrågade vädret. Om inga städer i Sverige har just det vädret, ska det komma en text som förklarar det. Texten ska även vara lite humoristisk.

Som det går att se i figur 3, försöker jag förstärka intrycket av en webbapplikation genom att göra i desktopläge så att kartan inte syns vid första anblick, och att fokuset ligger på logotypen och de fyra ikonerna. På ett sätt har jag en oro att det är ett dåligt val, då man inte ser hela webbplatsen direkt, men å andra sidan är sidan helt scrollbar och vid lyckad implementering tror jag att det blir en fin översikt från start.

3.4 Mashupen

För att klara av att göra min webbapplikation behöver jag en webbkarttjänst och ett API för att kunna hämta aktuellt väder. Dessa två måste fungera ihop och detta ska jag lösa genom att få dem att ”prata samma språk”. Kartan i sig har inte mycket funktion förutom att visa en karta och ofta namn på städer och API:n för väder är främst en ren databas bestående av bokstäver och siffror. Att kombinera dessa två möjliggör för en interaktiv tjänst som användare kan ha nytta av.

4 Teori

För att förstå min webbapplikation och dess förarbete och uppbyggnad, är det viktigt att känna till vissa begrepp som berör min process.

4.1 API

Ett API (Application Programming Interfaces) är ett slags osynligt hjälpmedel som hjälper till att föra över data från server till användaren med hjälp av en förfrågning (4). Genom att göra en förfrågning, en så kallad request, skickas förfrågan till en server som skickar tillbaka datan, för att det sedan ska levereras till slutanvändaren. Det finns olika typer av API, exempelvis webb-API som hämtar API från internet, lokala API där API:n kan finnas på en egen dator och interna API, som är API som kan finnas inom en viss organisation.

Fördelar med API är att det kan bidra till en innovativ integrering på en webbplats, som ofta även är automatiserad (4). Webbutvecklare kan välja vilken data man vill använda från ett API, men samtidigt bör man vara vaksam på att det finns risker med API. Ett API som hämtas från en extern leverantör kan sluta uppdateras, existera eller helt enkelt uppleva tekniska problem som i sin tur påverkar de som använder sig av dess API.

Trots riskerna, är API ett ofta använt sätt att utveckla en webbplats på, och möjligheterna är många.

4.2 Mashup

Genom att förstå nyttan av användning av API, kan man också se möjligheterna för att göra mashups. En mashup är ett vanligt sätt att utveckla en webbplats på och innebär i stora drag att man skapar en interaktiv webbplats som drar nytta av flera olika tjänster som finns att tillgå (5).

Det finns flera olika sätt att utforma en mashup, men ett vanligt sätt är att kombinera en karttjänst med någon typ av interaktionsmöjlighet (5). Ett välkänt exempel är Google Maps som kartjänst, även om det finns fler. Ett API kan här vara något som pekas ut på kartan med hjälp av digitala kartnålar. Man kan exempelvis visa vilka typer av evenemang som skett eller kommer ske på en specifik plats.

4.3 Async/await i JavaScript

För att hämta ett API kan en async function vara ett bra sätt att använda sig av (6). När en async function går igång, förväntas det returneras ett promise (6, 7). En promise kan i det stora hela jämföras med ett svar på en asyonkron funktion, som antingen är ett lyckat eller misslyckat svar. Ett exempel på ett promise är JSON-data, som kommer förklaras mer nedan.

Await hjälper koden att vänta på att den asynkrona funktionen ska få ett promise tillbaka, innan den fortsätter vidare i sin kod. På så sätt ser man till att önskad funktion i webbplatsen/webbapplikationen fungerar som den ska och inte hoppar över något. Detta är viktigt för de gånger när det kan ta en stund att få ett promise på sin förfrågan (6).

4.4 JSON

JSON är ett textformat för att lagra och överföra strukturerad data och arrayer mellan enheter (8).

Genom att hämta datan som ett JSON-format och översätta till JavaScript, kan man genom att utläsa datan och dess värden använda innehållet på sin webbplats, samtidigt som man lägger till funktioner eller design för slutanvändarna att ha nytta av datan (8).

4.5 Leaflet

Leaflet är ett JavaScript-bibliotek för kartor som möjliggör olika typer av interaktioner och plug-ins (9). Den är en enkel och användarvänlig tjänst med omfattande dokumentation som visar hur man kan använda sig av kartan, och den möjliggör även mashups med olika typer av API:er.

Leaflet visar även tutorials och hjälptexter för hur det kan fungera och vilka olika funktioner och lager man kan ha på sina kartor.

4.6 Open-Meteo

Open-Meteo erbjuder via öppen källkod en API-lösning för att visa väderdata på önskvärd plats (10). Webbutvecklare har möjlighet att göra ett flertal val för att kunna skräddarsy sin API i JSON-format och det går exempelvis att välja om man vill få fram information om solchanser, nederbörd, vind och molnighet, och man kan välja ett tidsintervall på upp till 16 dagars prognos. Väl när valen är gjorda, genereras en API-länk som i sin tur används i JavaScript-koden för att kunna programmera den till att passa ens önskemål och karttjänst.

4.7 CSS/SCSS och partials

Kort sagt är CSS det språk som används för att lägga till stil på ett HTML-dokument (11) och SCSS är ett preprocessoriskt stilspråk som är en förlängning av CSS med ännu fler funktioner (12). SCSS kallas för ett superset av CSS och möjliggör för utvecklare att skapa en mer strukturerad och återanvändbar kod. Exempelvis kan man använda sig av variabler; ofta för färger men även för andra värden som kan beröra teckenstorlek och olika typer av storlekar. Fördelen med variabler är att det är återanvändbar kod som dels innefattar koder på värden som används på en webbplats, och dels möjliggör färre upprepningar av kod. På så sätt ökar det chanserna för en enhetlighet på en webbplats.

Ett annat exempel på en SCSS-funktion är @extend som innebär att man kopierar en befintlig SCSS-kod och förlänger den (13). Fördelen med det är att två olika element i en kod kan ha nästan identisk stilkod, men med hjälp av @extend kopierar man en stilkod och lägger till de värden som man vill ha utöver.

Partials är också ett nytt sätt arbeta efter att SCSS börjat användas. Partials är bland annat bra för projekt där det kan vara svårt att hålla koll på all sin kod, och då delar man upp koden i olika komponenter för att lättare hitta vad man söker efter (14). Exempel på partials kan vara en för knappar, en för moduler, en för tema och så en huvudfil som använder sig av alla partials.

4.7.1 CSS-animationer

Det är möjligt att använda sig av animationer med hjälp av CSS och innebär att man gradvis ändrar ett utseende till ett annat med hjälp av keyframes (15).

Det är en fördel att ha en partial som samlar animationerna och det finns olika sätt att animera ett element (15). Ett exempel är att ändra färg på en knapp.

Det finns även transforms som gör en omvandlar ett objekt med hjälp av rotation, skalning och rörelse (16) medan transition kan förändra något och ofta vid hover eller klick, och under en viss tid (17).

4.8 JSDoc

JSDoc är ett verktyg för att kunna dokumentera JavaScript-kod med kommentarer direkt i koden (18). Detta för att kunna göra koden lättare att förstå, både för ens egen skull men även för de tillfällen när man samarbetar med andra.

5 Metod

5.1 Leaflet

Leaflet valde jag att använda eftersom jag använt det i en tidigare laboration och upplevt det som både enkelt att använda, men även för att dokumentationen och tutorials har och är väldigt hjälpsamma. Leaflet har ett gränssnitt som gör att jag bland annat kan sätta ut egna markörer i stället för den som finns som default. På så sätt har jag exempelvis haft möjlighet att ersätta default med egna ikoner för de olika väderlekarna.

5.2 Open-Meteo

Open-Meteo ger mig möjligheten att välja vilka specifika vädertyper och prognosdagar som jag vill kunna använda i min webbapplikation. Detta ger mig ett skräddarsytt JSON-format som jag sedan har möjlighet att koppla ihop med Leaflet.

5.3 Logotyp och ikoner

Eftersom det inte var ett krav på egenskapade ikoner i detta projekt, har jag lagt mindre tid än vanligt på att göra min egna ikoner som finns på startsidan när man ska välja önskat väder. I stället har jag använt mig av grafiska element från Canva till ikonerna och jag har blandat grafiska element från Canva för att göra min logotyp med hjälp av lager.

Även om de grafiska elementen är hämtade från Canva, så har jag haft som mål att ha en enhetlig design och utseende på ikonerna. Ikonerna har även återanvänts i kartan.

5.4 Grafisk profil

Jag har försökt hålla en blå färg som en röd tråd genom hela min webbapplikation. Den blå färgen finns i olika nyanser beroende på om det är en bakgrund, footer, hover-färg eller border. Den blå färg symboliserar en klar himmel och även om användarna har möjlighet att visa fler vädertyper än solklart, så heter sidan trots allt Solspaning och därför vill jag förknippa den grafiska profilen med en klar himmel.

5.5 SCSS

Jag använder mig av SCSS i min uppgift för att ha en bättre strukturerad arbetsgång.

5.5.1 Partials

De partials som jag ska ha med i min SCSS är animations, components, darkmode, structure, variables och huvudfilen styles. På så sätt ser jag till så att mina animationer samlas i en fil, mina komponenter och byggstenar i en fil, nattläge i en, huvudstruktur som främst berör body och html i en, samt variabler i den, där jag samlar färger som används i min webbapplikation.

5.5.2 Animationer

Jag använder mig av några animationer i min webbapplikation, men jag har försökt att hålla dem relativt ”lugna” och valt att inte göra något alldeles för galet eller stirrigt.

Jag kommer att ha animationer på texten under logotypen, och på ikonerna och de påhittade omdömena vid hover. I responsivt läge har jag lagt lite mer animation vid hover på ikonerna, exempelvis att ikonerna för sol och regn snurrar, men att ikonerna för vind och moln skakar lite.

5.5.3 Mörkt läge

Med hjälp av @media (prefers-color-scheme: dark) låter jag användare med mörkt läge på sin enhet se webbapplikationen på ett anpassat sätt, med svart bakgrund och vit text. Då jag anser att ikonerna funkar och syns bra i både ljust och mörkt läge modifierar jag inte dessa.

5.6 JavaScript

JavaScript används främst till för att få funktionerna med kartan och vädertjänsterna att fungera. Jag använder även JavaScript för att kunna skriva ut en text ifall vädret man söker efter inte finns just nu. Om jag inte gör detta, så blir kartan bara tom utan någon mer information och användaren vet inte om det ska tolka det som om vädret inte finns idag, eller om det är ett tekniskt fel eller fördröjning.

5.6.1 JSDoc

Jag använder mig av JSDoc för att dokumentera min JavaScript-kod. Genom att skriva dessa kommentarer ovanför mina funktioner kan jag beskriva vad funktionerna gör, vilka parametrar som tas emot och vad som returneras. Detta gör koden mer lättförståelig för mig och underlättar vid felsökning och eventuell vidareutveckling.

5.7 Git

Jag använder mig av GitHub för versionshantering och publicering, då min färdiga webbapplikation har fått en länk genom GitHub (19). Genom versionshanteringen använder jag mig av två brancher; development och main, varav den sistnämnda är kopplad till själva publiceringen.

6 Konstruktion

Följande underrubriker berör hur jag konstruerat min webbapplikation.

6.1 Webbapplikationens struktur

Min webbapplikation består av en HTML-sida. På denna sida finns en header och footer och innehåller däremellan är en body med main.

6.2 Kartan

Kartan har implementerats med hjälp av JavaScript-biblioteket Leaflet, där en karta initierades genom att jag skapade ett kartobjekt och valde att begränsa det till Sverige med hjälp av koordinater. För att kunna visa kartans innehåll lades ett kartlager från OpenStreetMap till. Mina anpassade markörer användes för att identifiera städer.

6.3 API-anrop

För att kunna hämta aktuell och relevant väderdata som jag filtrerat på Open-Meteos webbplats, gjordes API-anrop med hjälp av den genererade länken. Anropen gjordes med fetch () i kombination med async och await, som gjorde det möjligt att kunna vänta in svaret från API:et som skulle hämtas. Den information jag behövde få i mina anrop var väderkod, som gällde solen, nederbörd, vindhastighet och molnighet, men även om det är natt eller dag.

När jag fick tillbaka svaret omvandlades det från ett JSON-format till ett JavaScript-objekt, och på så sätt kunde jag använda mina villkor för att avgöra om och var en markör skulle placeras ut på kartan. Ett exempel på kod för soligt väder är:

```
async function showSunnyWeather(city) {

  const url = `https://api.open-meteo.com/v1/forecast?latitude=${city.lat}&longi-
  tude=${city.lon}&current=precipitation,weather_code,wind_speed_10m,cloud_co-
  ver,is_day&forecast_days=1&wind_speed_unit=ms`;

  const response = await fetch(url);

  const data = await response.json();

  if (data.current.weather_code <= 2 &&

    data.current.cloud_cover <= 50 &&

    data.current.precipitation === 0 &&

    data.current.is_day === 1) {

    L.marker([city.lat, city.lon],

      { icon: sunnyIcon }).addTo(marker)

      .bindPopup("Solen skiner här i " + city.name + "!");

  }
```

```
  else if (
    data.current.weather_code <= 2 &&
    data.current.is_day === 0
  ) {
    L.marker([city.lat, city.lon],
      { icon: moonIcon }).addTo(marker)
      .bindPopup("Det är klart väder i " + city.name + ", men solen har gått ner");
  }
}
```

I detta kodexempel har jag först genererat länken till Open-Meteos API, där stadens koordinater finns med som parametrar. Därefter använder jag en fetch för att skicka en förfrågan till tjänsten och med hjälp av await väntar man först på att få svar från servern och sedan på att svaret omvandlas till JSON. När datan sedan har hämtats så använder jag mig av villkor för att kontrollera om vädret ska räknas som soligt, enligt data som finns på Open-Meteos sida (10). Om villkoren uppfylls, så placeras en markör ut på kartan tillsammans med en ikon och informationsruta.

6.4 Logiken

För att göra min webbapplikation interaktiv så har jag använt ”klickhändelser” som är kopplade till de olika väderikonerna. När användaren klickar på en ikon under logotypen, det vill säga sol, regn, vind eller moln, så ska det motsvara vädret man önskar ha och då aktiveras en funktion för vad som ska hända härnäst. Då jag har en array med femton städer vars namn och koordinater jag skrivit ut, görs en for of-loop där det för varje stad anropas en funktion som hämtar väderdata från API:et och sedan avgör om stadens väder matchar användarens önskemål. På detta sätt ser jag till att de städer som finns på kartan har rätt väder som visas på ikonerna.

Som ett exempel på detta så har jag funktionen showSunnyCities som först rensar kartan från tidigare markörer och sedan loopar igenom listan med städer. För varje stad anropas funktionen showSunnyWeather (se punkt 6.3). Genom att använda await här också så ser jag till att alla API-anrop hinner slutföras innan nästa körs. På så sätt syns rätt väder på kartan. Utdrag ur koden för detta ser i mitt fall ut såhär:

```
async function showSunnyCities() {
 marker.clearLayers();
  const element = document.getElementById("map");
```

```
  const message = document.getElementById("weatherText");

  message.textContent = "";

  for (const city of cities) {

    await showSunnyWeather(city);

  }

}
```

6.5 UX

Eftersom jag ville ha ett webbapplikationslikt utseende så har jag behövt både ge och ta gällande min design. Att välja att endast visa logotyp och de fyra väderikonerna som täcker skärmen, se figur 3, så har jag också skärmvisuellt dölja resterande del av webbplatsens innehåll. Med detta menar jag att kartan, som är en väsentlig del av webbapplikationen och dess syfte, inte syns såvida man antingen trycker på en av ikonerna, eller väljer att scrolla ned. Därför ville jag att man nästan ”tvingas” klicka på en ikon när man kommer in på webbplatsen, vilket också är ett stort syfte med denna, och vid klick har jag då valt att man automatiskt scrollas ner till kartan. I kartan visas då alltså ikoner med det vädret man önskar, utplacerade på de städer i Sverige (baserat på min förinlagda array) som har detta väder just nu.

Jag har under arbetets gång märkt att det flera gånger hänt att vädret man önskar för den stunden inte finns i något av städerna. Att ha en tom karta kan vara missvisande och jag själv visste inte om det var ett tekniskt fel eller att vädret saknades i landet just då.

Jag valde då att ha ett if-villkor att om villkoret inte uppfylldes, så visades en text, och annars scrollades man ner till kartan. Detta gjorde också att det blev tydligare att se att funktionen fungerade och att det inte var något tekniskt fel. Kod som jag använt för detta:

```
async function showSunnyCities() {

 marker.clearLayers(); //för att rensa fält innan nästa klick

  const element = document.getElementById("map");

  const message = document.getElementById("weatherText");

  message.textContent = "";

  for (const city of cities) {

    await showSunnyWeather(city);

  }

  if (marker.getLayers().length === 0) {

    message.scrollIntoView({
```

```
        behavior: "smooth"

      });

      message.textContent = "Varning för brist på vitamin D idag! Men att sitta inomhus och titta på en film är också bra för hälsan ibland!";

   }

   else {

      element.scrollIntoView({

      behavior: "smooth"

   });

   }

}
```

Genom att använda denna kod, rensas först markörerna från kartan, och därefter loopas alla städer igenom och vädret kontrolleras, vilket medför att ikoner placeras ut. Om inga ikoner placeras ut, ska i stället en text skrivas ut enligt villkoret.

6.6 SCSS

I stället för vanlig CSS har jag använt mig av SCSS och några av dess många funktioner. Jag har använt mig av olika partials för att kunna strukturera upp min kod och lättare identifiera eventuella fel eller saker jag velat ändra. Exempel på partials jag använt mig av är struktur, komponenter och animationer. Att använda mig av partials har varit fördelaktigt då jag både har en översikt över min kod, men även gör koden möjlig för återanvändning i andra projekt framöver.

För färg användes variables och detta hjälpte mig att hålla en enhetlighet gällande den grafiska profilen. Genom att ändra en färg på ett ställe, har jag möjlighet att få ändringen att slå igenom på hela webbplatsen om jag så önskar.

Gällande animationer så har jag valt att göra innehållet i webbapplikationen mer levande, och jag har främst valt att ha animationer som triggas igång av hover. Mitt mål är att animationerna ska vara lagom och varken förvirrande eller störande.

Slutligen har jag använt mig av media query prefers-color-scheme: dark som gör att webbapplikationen anpassar utseendet baserat på användarens inställningar och där jag gjort inställningar i min SCSS-kod för att se till så att det inte finns svart text på svart bakgrund eller att det finns andra tillgänglighetsproblem när bakgrunden blir mörk.

7 Resultat

Resultatet av mitt projekt har blivit sidan Solspaning, där användaren utifrån humör eller önskemål kan välja mellan olika vädertyper och på en karta se de fördefinierade svenska städerna som för tillfället har det efterfrågade vädret.

Min webbapplikation använder sig av Open-Meteo för att hämta aktuell väderdata och Leaflet för att visualisera väderdatan på en interaktiv karta. Varje vädertyp är baserade på olika data och vid exempelvis klart väder visas en sol-ikon på kartan. Om ingen stad uppfyller det valda vädervillkoret så visas i stället en kort text ovanför kartan, vilket ger användaren en återkoppling oavsett om vädret finns eller ej.

Jag har använt mig av en tydlig grafisk profil, animationer och stöd för mörkt läge för de som har den inställningen på sin enhet. Jag har vidare strukturerat min kod med SCSS och partials, och använt mig av Git och Github för att versionshantera och publicera webbapplikationen.

Resultatet uppfyller kraven då jag kombinerar väderdatan och visualiseringen i kartan, till en interaktiv webbapplikation med responsiv design.

8 Slutsatser

En insikt jag har haft under hela projektets gång är att väderdata kan tolkas och användas på olika sätt. För att avgöra om ett väder skulle räknas som exempelvis soligt eller molnigt så behövde jag själv bestämma villkoren. Dessa har också förtydligats på själva webbapplikationen i form av kort, förklarande text.

Mina val kan självklart påverka resultatet och mina tankar och tolkningar kan skilja sig från användarnas. Min lösning är därför inte den enda korrekta eller möjliga, men den är konsekvent.

Om projektet hade vidareutvecklats med ännu mer kompetens och tid, hade det varit intressant att låta användaren välja fler städer, justera väderdatans gränser och använda flera datakällor för att lägga till ännu fler funktioner.

9 Källförteckning

[1] GeeksforGeeks, “What is a mashup in web technology?”,
https://www.geeksforgeeks.org/websites-apps/what-is-a-mashup-in-web-technology/
Publicerad 2025-07-23. Hämtad 2026-03-20

[2] Klart.se,
https://www.klart.se/
Hämtad 2026-03-210

[3] TV4-vädret,
https://www.tv4play.se/program/5c99bd3a3ec4ae952052?utm_source=tv4.se&utm_medium=Hero&utm_campaign=upsell
Hämtad 2026-03-210

[4] GeeksforGeeks, ”What is an API (Application Programming Interface)”,
https://www.geeksforgeeks.org/software-testing/what-is-an-api/
Publicerad 2025-12-15. Hämtad 2026-03-20

[5] Medium, ”Mashups: What Are They? Mashup Genres And Technologies – Part 1”,
https://masternewmedia.com/mashups-what-are-they-mashup-genres-and-technologies-part-1-bc9df6b36986#2266
Publicerad 2024-01-09. Hämtad 2026-03-20

[6] Mozilla Developer, ”async function”,
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
Hämtad 2026-03-21

[7] W3Schools, ”JavaScript Promises”,
https://www.w3schools.com/JS//js_promise.asp
Hämtad 2026-03-21

[8] W3Schools, ”JavaScript JSON”,
https://www.w3schools.com/js/js_json.asp
Hämtad 2026-03-21

[9] Leaflet, ”an open-source JavaScript library for mobile-friendly interactive maps”,
https://leafletjs.com/
Hämtad 2026-03-21

[10] Open-Meteo, ”Free Weather API”,
https://open-meteo.com/
Hämtad 2026-03-21

[11] W3Schools, ”CSS Tutorial”,
https://www.w3schools.com/css/
Hämtad 2026-03-21

[12] GeeksforGeeks, ”Difference Between CSS and SCSS”,
https://www.geeksforgeeks.org/css/what-is-the-difference-between-css-and-scss/
Publicerad 2025-01-15. Hämtad 2026-03-21

[13] W3Schools, ”Sass @extend and Inheritance”,
https://www.w3schools.com/sass/sass_extend.asp
Hämtad 2026-03-21

[14] Dev, ”Using SASS partials”,
https://dev.to/sarah_chima/using-sass-partials-7mh
Publicerad 2017-11-13. Hämtad 2026-03-21

[15] W3Schools, ”CSS Animations”,
https://www.w3schools.com/css/css3_animations.asp
Hämtad 2026-03-22

[16] W3Schools, ”CSS 3D Transforms”,
https://www.w3schools.com/css/css3_3dtransforms.asp
Hämtad 2026-03-22

[17] W3Schools, ”CSS Transitions”,
https://www.w3schools.com/css/css3_transitions.asp
Hämtad 2026-03-22

[18] Medium, ”A Comprehensive Guide to JSDoc Comments in JavaScript”,
https://medium.com/@uomroshan/a-comprehensive-guide-to-jsdoc-comments-in-javascript-ed14df2351ef
Publicerad 2024-06-24. Hämtad 2026-03-22

[19] Solspaning,
https://elro2506.github.io/projektuppgift/
Hämtad 2026-03-22

10 Bilagor

Figur 1: Figma-prototyp i desktopläge

Figur 2: Figma-prototyp i responsivt läge

Figur 3: Utklipp ur Figma-prototyp som visar önskad vy vid desktopläge innan scroll